

Distributed Databases using Replication

Homework 10 Report

Sampath Pranay Beela

CS6650 — Building Scalable Distributed Systems

1 Problem, Repository, and Overview of Experiments

This homework implements an in-memory key-value store in two distributed forms: a leader-follower database and a leaderless database. Both expose the same two client operations, `set(key, value)` and `get(key)`, but they make different tradeoffs around replication, read latency, write latency, and stale-read exposure.

The repository for this submission is <https://github.com/beelapranay/CS6650-BSDS.git>. The homework code for this report lives under `hw10/` and includes:

- Leader-follower service code
- Leaderless service code
- Dockerfiles and Compose configurations
- Unit tests for consistency behavior
- Load-test client and graph generator
- Generated result JSON files under `results/`
- Generated graphs under `graphs/`

The experimental plan matches the assignment requirements. Four workloads were run for each of the three leader-follower configurations and the leaderless configuration:

- 1% writes / 99% reads
- 10% writes / 90% reads
- 50% writes / 50% reads
- 90% writes / 10% reads

For each run, the load tester recorded request latencies, client-side stale-read counts, and the time interval between a write and the next read of the same key. The graphs in this report are generated directly from those result files.

2 Architecture

The system has two independent implementations.

Leader-follower. There are five nodes total: one leader and four followers. All writes go to the leader. The leader updates its own local store, assigns a logical version number, and then replicates the update to followers according to the configured write quorum W . Reads are handled according to the configured read quorum R . For $R=1$, the leader reads locally. For larger R , the leader queries followers using an internal read endpoint and returns the value with the highest version.

Leaderless. There are five equal peers. Any node can receive a client write and becomes the write coordinator for that request. The coordinator writes locally, propagates the write to the other four nodes, waits for all acknowledgments because $W=N$, and then returns to the client. Reads are local because $R=1$.

Both implementations are purely in memory, so all data is lost when containers stop. This is intentional and matches the homework specification.

The architecture can be summarized as follows:

Leader-Follower:

```
Client --> Leader /set --> local write + version bump --> follower replication
Client --> Leader /get --> local read or quorum read --> most recent version
```

Client --> Follower /local_read --> local state only (testing endpoint)

Leaderless:

Client --> Any node /set --> local write + version bump --> replicate to peers

Client --> Any node /get --> local read only

3 How the Code Works

Leader-Follower Cases

W=5, R=1. This configuration waits for every follower update before the leader acknowledges a write. In practice, writes are expensive because the leader sends four replication messages, sleeps 200 ms after each, and each follower sleeps 100 ms before acknowledging. The measured write latency of roughly 1.2–1.3 seconds matches that design. Reads are cheap because the leader returns its local value immediately.

W=1, R=5. This configuration optimizes for writes. The leader updates itself and returns immediately, while replication to followers continues in a background thread. Reads are expensive because the leader must gather values from all replicas and return the newest version. This is why the measured write medians are only a few milliseconds, while read medians are around 215–250 ms. This configuration deliberately exposes a wide inconsistency window.

W=3, R=3. This quorum configuration is the middle ground. A write only needs the leader plus two followers, and a read only needs the leader plus two follower responses. The measured latencies reflect exactly that compromise: writes are much slower than W=1, but materially faster than W=5; reads are slower than R=1, but materially faster than R=5.

Leaderless Case

The leaderless service uses **W=N, R=1**. A write coordinator must reach all nodes before acknowledging the client, so writes are again around 1.2–1.4 seconds. Reads are local and therefore fast, but a read that lands on a node that has not yet received a new version can still observe stale state during the propagation window. This is the core tradeoff the leaderless design demonstrates.

Versioning, Delays, and Tricky Parts

Logical versions are attached to every key-value pair. The services use those version numbers to decide which replica has the newest value and to avoid overwriting a newer version with an older one.

The most important implementation details are:

- The leader sleeps 200 ms after each follower replication message.
- Followers sleep 100 ms before acknowledging a replicated write.
- Followers sleep 50 ms before answering an internal quorum read.
- `local_read` is an intentionally unsafe endpoint used only to reveal inconsistency during tests.

If I were handing this code to another engineer, the main warning would be that the measured latency numbers are not accidental. They are a direct consequence of the artificial delays, serial replication loops, and serial follower read loops. The code is simple, but the simple control flow produces very visible performance signatures.

Operational Notes and Error Handling

If I were handing this project to a friend to maintain, these are the first things I would tell them.

- This is an in-memory system only. Restarting containers clears all keys, so any test that depends on prior state must recreate that state after startup.
- The leader-follower service only accepts client writes on the leader. Sending `/set` to a follower returns 403.
- Empty keys are rejected with 400. Empty values are accepted, which is required by the assignment.
- Reads for missing keys return 404. This is true for normal reads and for `local_read`.
- If the leader-follower service cannot collect enough acknowledgments to satisfy W , it returns 500. If the leaderless coordinator cannot reach all peers for $W=N$, it also returns 500.
- Replication is version-based. A node only overwrites its local value if the incoming version is newer. That rule prevents old updates from replacing new ones when requests overlap.
- The code currently performs replication and quorum reads sequentially rather than in parallel. That keeps the code easy to reason about, but it also amplifies the effect of the artificial delays.
- The $W=1, R=5$ path is the easiest place to make incorrect claims when presenting results. Writes are fast because replication is asynchronous, but the temporary inconsistency that follows is a feature of the design, not a bug in the measurements.

Because this is an individual submission, I made sure I can explain each endpoint, each quorum case, each test, and each graph directly from the code and the measured outputs.

4 Testing Methodology

Two categories of validation were used.

Unit tests. The repository includes:

- `tests/test_leader_follower.py`
- `tests/test_leaderless.py`

The leader-follower tests validate successful write-followed-by-read behavior and also use `local_read` to expose stale follower state during the update window. During local validation, the leader-follower suite completed successfully and reported the expected inconsistency signal during the race-window probe.

The leaderless suite performs the analogous checks for $W=N, R=1$. On macOS, host port 7000 was already occupied by a system process, so the leaderless host bindings were remapped to ports 7100–7104 for this submission. The compose file, tests, and load tester were updated together so that the environment remains internally consistent.

Load tests. The script `run_all_tests.sh` executes all four workloads against:

- leader-follower $W=5, R=1$
- leader-follower $W=1, R=5$
- leader-follower $W=3, R=3$
- leaderless $W=5, R=1$

The load tester deliberately uses only 10 keys. This creates key locality in time: the same keys are read and written repeatedly in a short period, which increases the chance of revealing stale reads and generates meaningful read/write interval distributions. That design choice directly addresses the assignment requirement that reads and writes cluster around the same key.

5 Measured Results

Table 1 summarizes the generated JSON result files.

Table 1: Load-test summary from `results/*.json`

Config	Writes	Throughput	Stale reads	Write med.	Write p95	Read med.	Read p95
w5r1	1%	327.88 req/s	0.0%	1244.35 ms	1322.75 ms	3.63 ms	10.34 ms
w5r1	10%	78.08 req/s	0.0%	1243.07 ms	1273.55 ms	2.37 ms	12.74 ms
w5r1	50%	14.81 req/s	0.0%	1286.71 ms	1352.09 ms	8.38 ms	25.09 ms
w5r1	90%	8.36 req/s	0.0%	1295.16 ms	1326.95 ms	15.29 ms	24.59 ms
w1r5	1%	40.40 req/s	0.0%	9.63 ms	10.38 ms	248.15 ms	269.69 ms
w1r5	10%	46.82 req/s	5.7%	6.68 ms	13.50 ms	229.85 ms	255.42 ms
w1r5	50%	90.29 req/s	52.7%	3.59 ms	18.59 ms	215.01 ms	229.74 ms
w1r5	90%	291.81 req/s	98.0%	3.59 ms	17.09 ms	215.76 ms	479.77 ms
w3r3	1%	42.95 req/s	0.0%	624.96 ms	632.14 ms	236.55 ms	260.72 ms
w3r3	10%	52.39 req/s	0.0%	622.02 ms	639.26 ms	119.87 ms	242.94 ms
w3r3	50%	27.61 req/s	0.0%	623.33 ms	637.96 ms	118.09 ms	130.30 ms
w3r3	90%	16.80 req/s	0.0%	631.08 ms	649.59 ms	122.21 ms	133.22 ms
leaderless	1%	350.74 req/s	0.0%	1238.83 ms	1256.32 ms	2.13 ms	7.16 ms
leaderless	10%	77.15 req/s	0.0%	1235.52 ms	1280.63 ms	3.34 ms	14.31 ms
leaderless	50%	16.12 req/s	0.0%	1274.91 ms	1335.65 ms	8.64 ms	23.10 ms
leaderless	90%	8.77 req/s	0.0%	1286.26 ms	1367.08 ms	12.61 ms	23.95 ms

The read/write interval medians also show the intended key locality effect. In the read-heavy runs, intervals were much larger because writes were rare. In the mixed and write-heavy runs, intervals dropped sharply, often to a few tens of milliseconds for `w5r1` and `leaderless`, which means the client was repeatedly revisiting recently written keys.

6 Discussion of Results

Which leader-follower type does best for each read/write ratio?

1% writes / 99% reads. Among the leader-follower options, `W=5, R=1` is the best fit. Its write path is expensive, but writes are rare, and reads stay extremely cheap because they are leader-local. `W=1, R=5` and `W=3, R=3` both waste too much time on follower reads in a workload dominated by reads.

10% writes / 90% reads. `W=5, R=1` still wins among the leader-follower options. The workload is still read-heavy enough that fast reads dominate the overall throughput.

Latency Distributions - w5r1

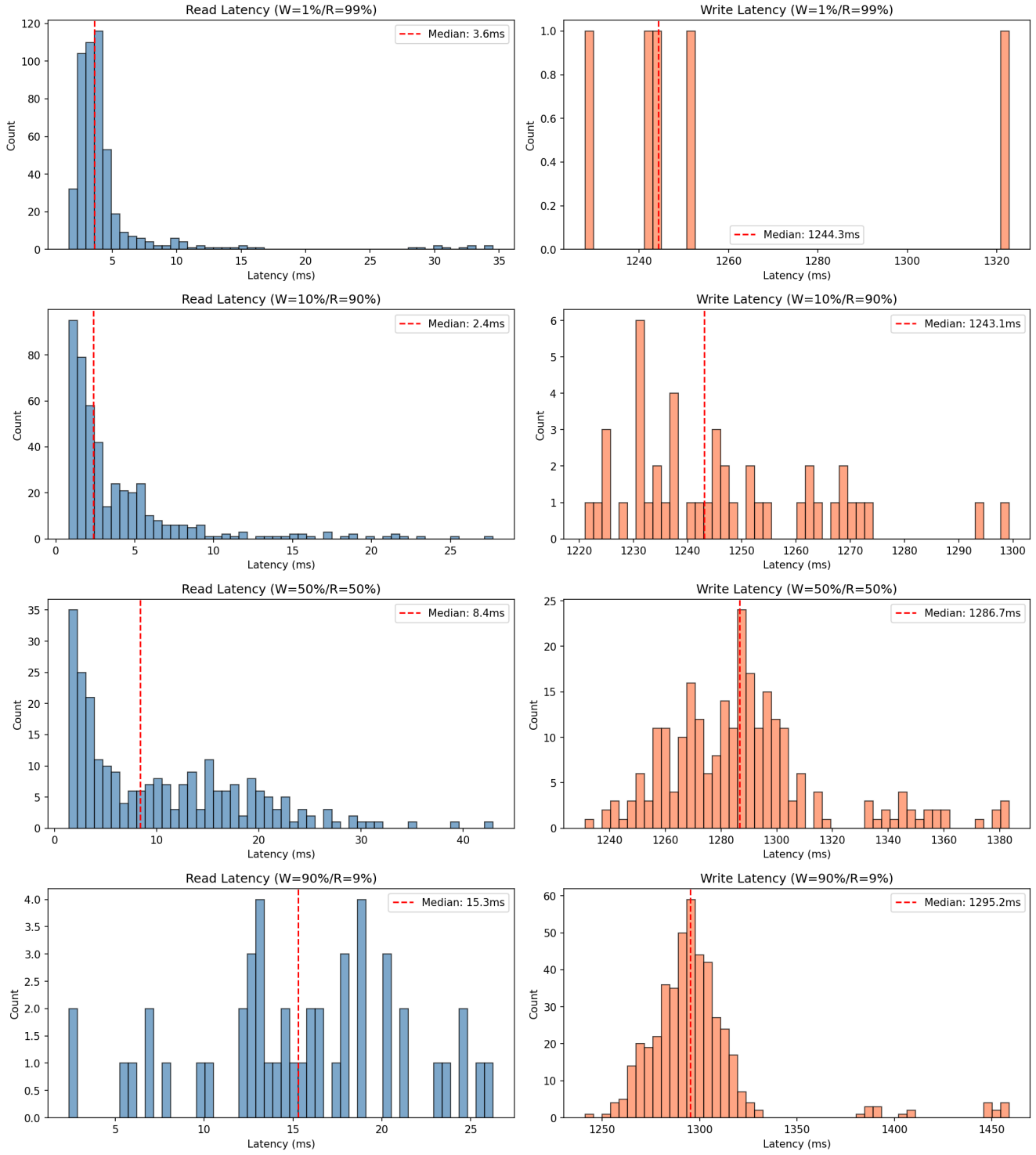


Figure 1: Latency distributions for leader-follower W=5, R=1.

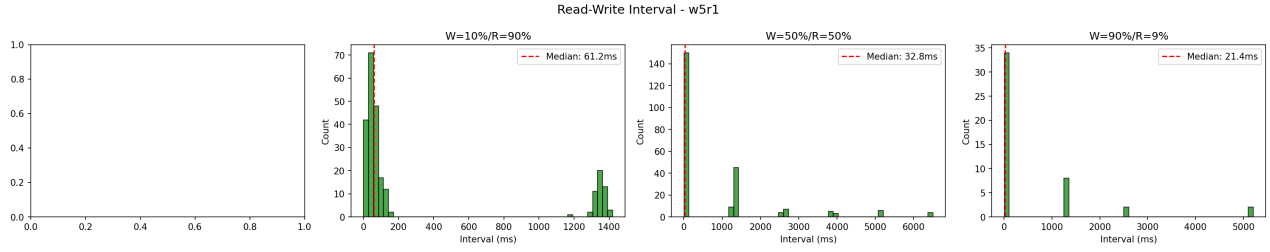


Figure 2: Read/write interval distributions for leader-follower $W=5$, $R=1$.

Figures 1 and 2 match that interpretation. Read latency stays tightly clustered near the floor of the service, while write latency forms the expected long cluster around 1.2–1.3 seconds because every write waits for all four followers.

50% writes / 50% reads. The answer depends on what “best” means. If the goal is pure throughput, $W=1$, $R=5$ is the fastest because writes are almost free. If the goal is a balanced consistency/performance tradeoff, $W=3$, $R=3$ is the better leader-follower choice because it avoids the very high stale-read rate and still cuts both read and write costs relative to the two extremes.

90% writes / 10% reads. $W=1$, $R=5$ is clearly best for throughput because the system mostly pays the cheap write path and only occasionally pays for a quorum read. The downside is exactly what the homework predicts: a very large inconsistency window.

Latency Distributions - w1r5

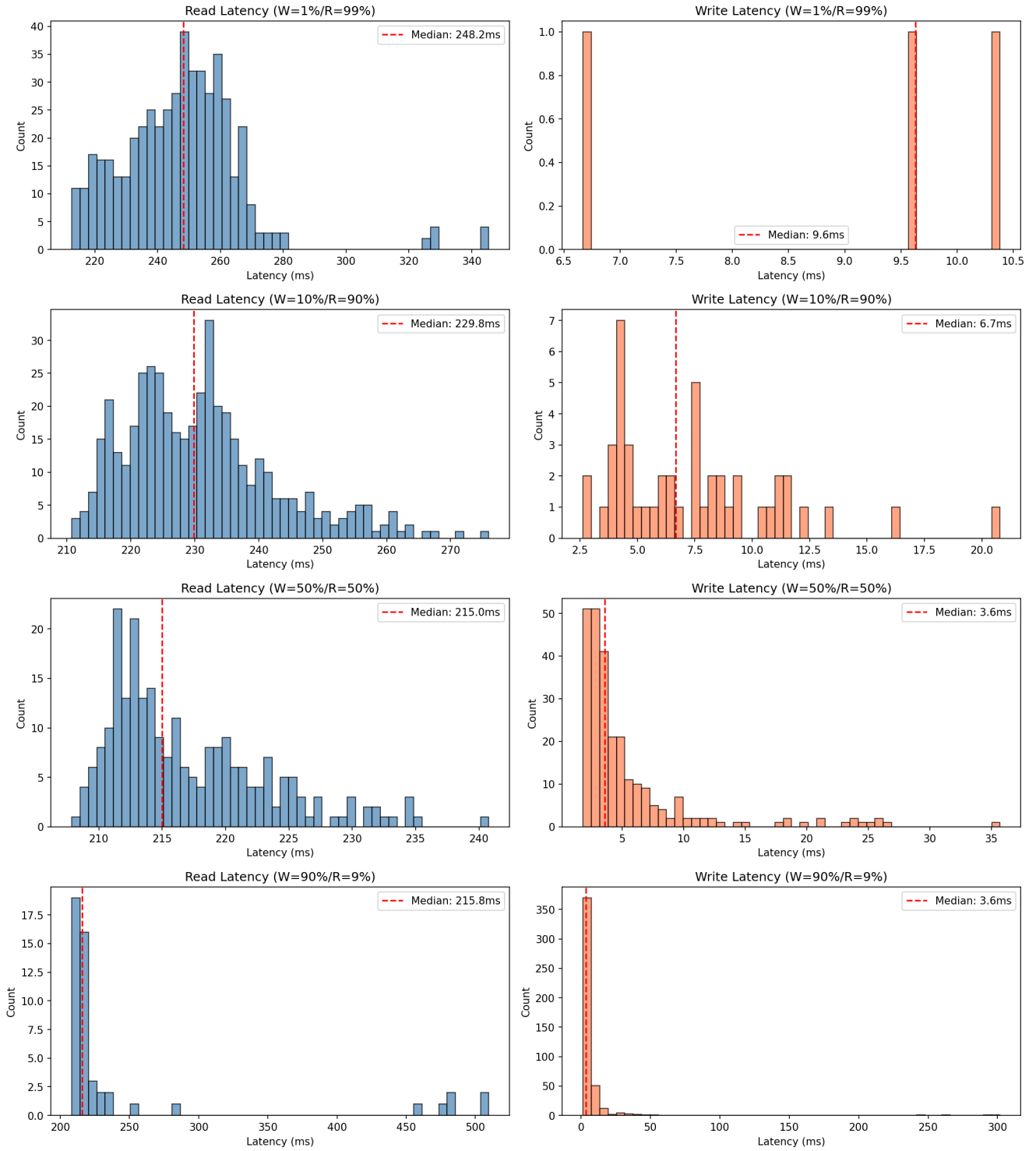


Figure 3: Latency distributions for leader-follower $W=1$, $R=5$.

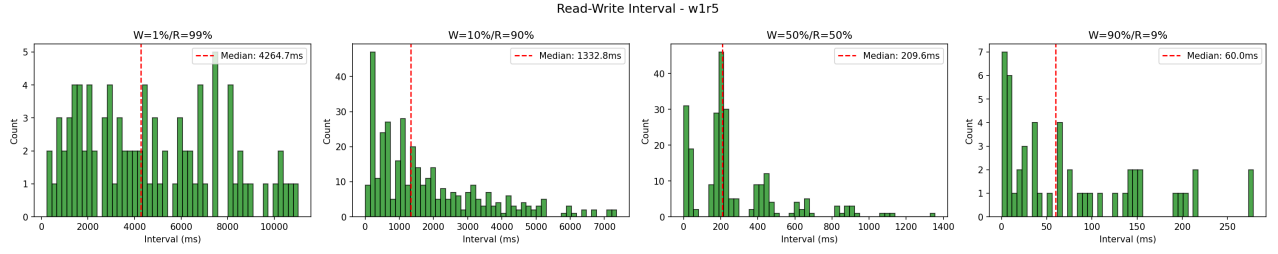


Figure 4: Read/write interval distributions for leader-follower $W=1$, $R=5$.

Figures 3 and 4 make the asymmetry of $W=1$, $R=5$ obvious. The write histogram sits in the low-millisecond range, but the read histogram shifts far to the right because each read pays for the leader's sequential quorum collection. The interval plot also shows that when reads and writes touch the same key close together, this configuration is exactly where stale visibility is most likely.

Latency Distributions - w3r3

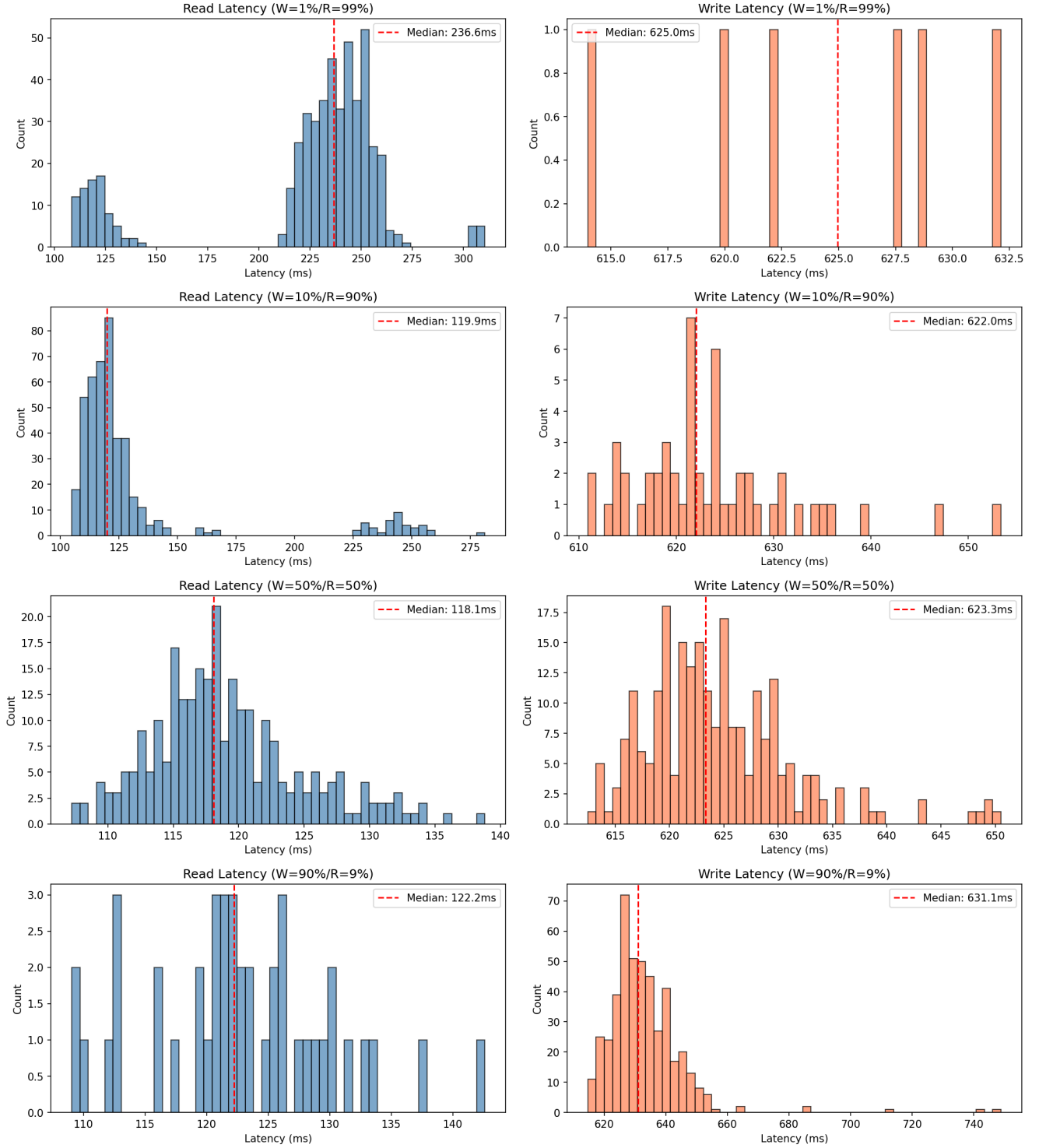


Figure 5: Latency distributions for leader-follower W=3, R=3.

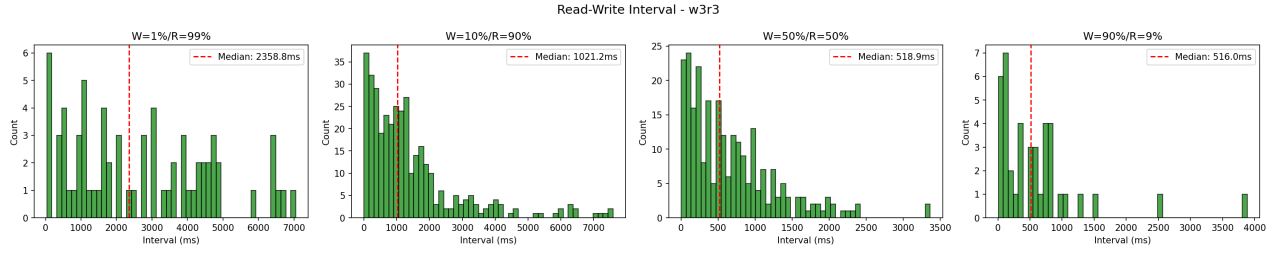


Figure 6: Read/write interval distributions for leader-follower $W=3$, $R=3$.

Figures 5 and 6 show the intended middle-ground behavior. Both read and write distributions shift inward relative to the two extremes: reads are much cheaper than $R=5$, and writes are much cheaper than $W=5$, but neither operation is as cheap as in the single-sided optimized designs.

Why the numbers look the way they do

The results align closely with the implementation structure.

- $W=5$, $R=1$ and leaderless have nearly identical write latency because both wait for all four remote updates. The artificial delay budget alone is roughly $4 \times (200 \text{ ms coordinator sleep} + 100 \text{ ms replica processing})$, so a write should land near 1.2 seconds before network overhead.
- $W=1$, $R=5$ has extremely fast writes because the leader returns immediately after updating itself. Its slow reads are expected because the leader performs sequential follower reads and each follower sleeps 50 ms first.
- $W=3$, $R=3$ lands almost exactly between the two extremes. The write path only needs two follower acknowledgments beyond the leader, and the read path only needs two follower replies beyond the leader.
- Leaderless behaves similarly to $W=5$, $R=1$ in latency because it also waits for all nodes on writes and uses local reads.

One especially important observation is that the latency curves are driven more by replication policy than by contention on the in-memory store. The service logic is simple; the protocol choice is what dominates the timings.

Latency Distributions - leaderless

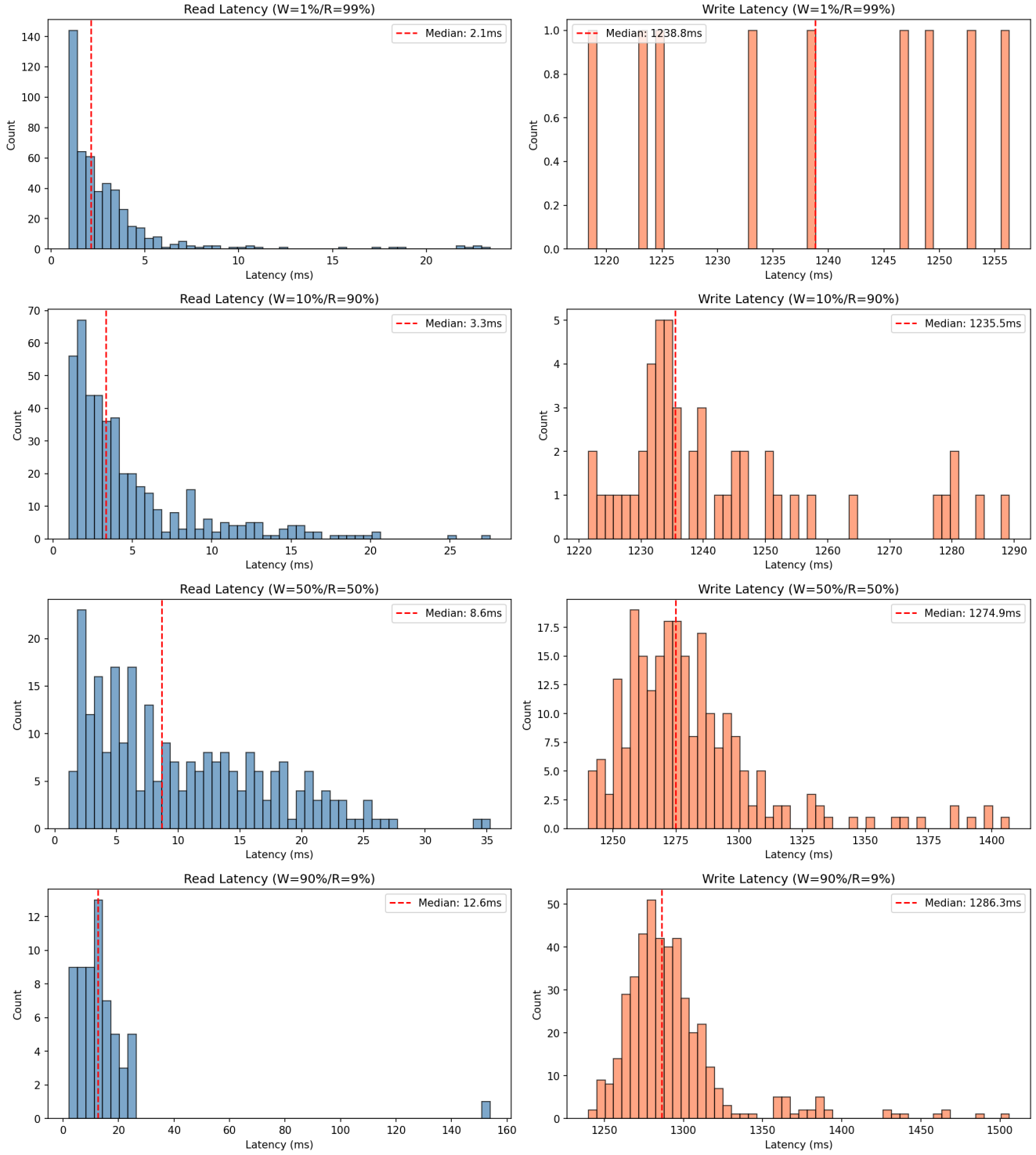


Figure 7: Latency distributions for the leaderless configuration $W=5$, $R=1$.

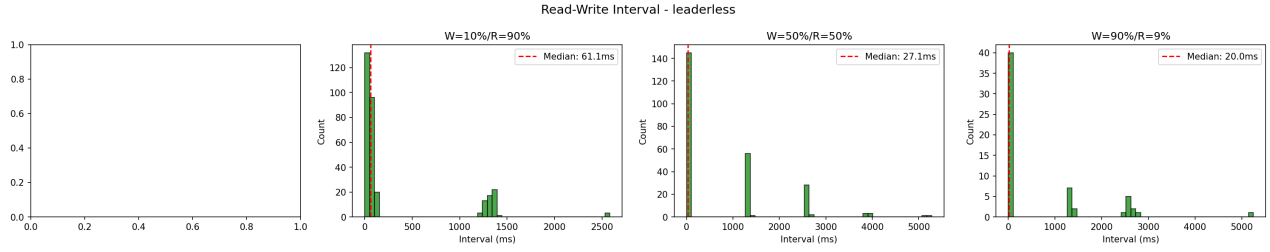


Figure 8: Read/write interval distributions for the leaderless configuration.

Figures 7 and 8 show why the leaderless case resembles $W=5$, $R=1$ so closely in latency but not in protocol structure. The write coordinator still waits for all peers, so writes remain expensive, while reads are local and therefore stay fast until they encounter propagation delay.

Which database is best for which application?

$W=5$, $R=1$. Best for read-heavy applications that want strong post-acknowledgment consistency and can tolerate expensive writes. A catalog service or configuration service fits this pattern.

$W=1$, $R=5$. Best for write-heavy workloads where write latency matters more than temporary replica freshness. It is attractive when clients mostly write and only occasionally pay for a read repair style operation.

$W=3$, $R=3$. Best for balanced workloads that want a more symmetric consistency/performance tradeoff without paying the full cost of all-replica acknowledgments on every request.

Leaderless $W=N$, $R=1$. Best when there is no distinguished leader and fast local reads matter, but the application can tolerate a short inconsistency window during propagation. This is useful for decentralized systems or topologies where a fixed leader is undesirable.

Limitations of the stale-read metric

The stale-read summary should be interpreted carefully. The leader-follower load tester currently sends client reads to the leader and then compares the returned version against a client-side “latest seen” version under concurrent activity. That means the $W=1$, $R=5$ stale percentages are still useful as a signal that asynchronous replication creates visibility gaps, but they are not a perfect direct measurement of follower stale reads. The unit-test `local_read` probes are the cleaner proof that stale replica state exists during the update window.

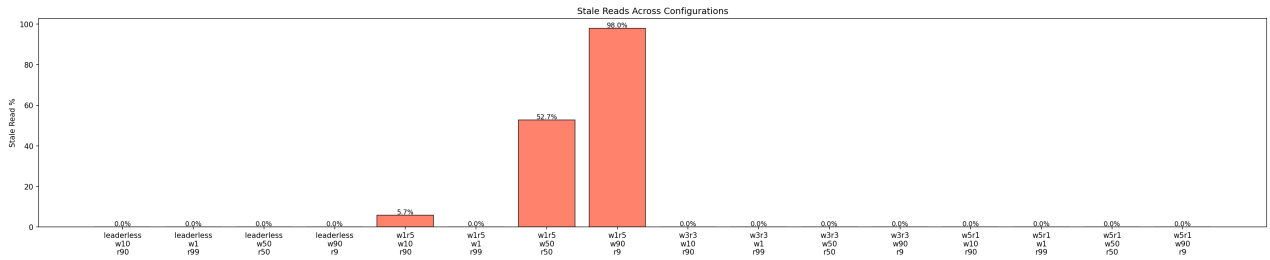


Figure 9: Client-side stale-read summary across all configurations and workloads.

Figure 9 is placed here because it belongs with the caution about interpretation. It is still useful as a comparative artifact, especially for showing that asynchronous $W=1$, $R=5$ behavior creates far more stale visibility than the other runs, but it should be discussed alongside the methodology limitation rather than as an isolated graph dump.

7 Submission Checklist

This submission includes the code, configurations, tests, load generator, and graphs required by the assignment. The most important files are:

- leader-follower/app.py
- leaderless/app.py
- tests/test_leader_follower.py
- tests/test_leaderless.py
- load-tester/load_test.py
- load-tester/generate_graphs.py
- docker-compose-*.yaml
- run_all_tests.sh

8 Conclusion

This homework makes the tradeoffs among replication strategies very concrete. $W=5$, $R=1$ delivers excellent read performance with expensive writes, $W=1$, $R=5$ flips that tradeoff and creates a large inconsistency window, and $W=3$, $R=3$ behaves as a true middle ground. The leaderless implementation shows that removing a distinguished leader does not eliminate the cost of strong writes; it simply moves the coordination role to whichever node receives the request.

The measured data strongly supports the course material: replication policy, quorum design, and when the system chooses to wait dominate both latency and consistency behavior.